

Formal Languages, Fall 2008

Homework 4 Reference Solutions

4.12 $A = \{ \langle R, S \rangle \mid R \text{ and } S \text{ are regular expressions and } L(R) \subseteq L(S) \}$.

We observe that $L(R) \subseteq L(S)$ if and only if $\overline{L(S)} \cap L(R) = \emptyset$. The following TM X decides A .

$X =$ "On input $\langle R, S \rangle$ where R and S are regular expressions:

1. Construct DFA E such that $L(E) = \overline{L(S)} \cap L(R)$.
2. Run TM T on $\langle E \rangle$, where T decides EQ_{DFA} from Theorem 4.4.
3. If T accepts, *accept*. If T rejects, *reject*."

Since T will always halt, X will always halt and make a decision. Hence, A is decidable.

4.19 $S = \{ \langle M \rangle \mid M \text{ is a DFA that accepts } w^R \text{ whenever it accepts } w \}$.

For any language A , let $A^R = \{w^R \mid w \in A\}$, observe that if $\langle M \rangle \in S$ then $L(M) = L(M)^R$. The following TM T decides S .

$T =$ "On input $\langle M \rangle$, where M is a DFA:

1. Construct DFA N that recognizes $L(M)^R$ using the following construction.
 - (a) A NFA N' is constructed from M firstly.
 - (b) Keep all states in M and reverse the direction of all arrows in M .
 - (c) Set the start state in M as the accept state in N' .
 - (d) Create a new state as the start state in N' and introduce new arrows, i.e. ϵ -transitions, from the start state to every accept state in M .
 - (e) Transform the NFA N' to a DFA called N , which recognizes $L(M)^R$.
2. Run TM F from Theorem 4.5 on $\langle M, N \rangle$, where F is the Turing machine deciding EQ_{DFA} ."
3. If F accepts, *accept*. If F rejects, *reject*.

Since EQ_{DFA} is decidable, then F will always halt and make a decision. T always halts and makes a decision on the input and thus S is decidable.

4.26 $C = \{ \langle G, x \rangle \mid G \text{ is a CFG that generates some string } w, \text{ where } x \text{ is a substring of } w \}$.

The following TM M decides C .

$M =$ "On input $\langle G, x \rangle$ where G is a CFG:

1. Let the regular expression $E = \Sigma^* x \Sigma^*$.
2. Construct a CFG H such that $L(H) = L(E) \cap L(G)$.
3. Run TM R from Theorem 4.8 on input $\langle H \rangle$.

4. If R accepts, reject; otherwise, accept.”

Note that $L(H)$ is a context-free language because $L(E)$ is regular and $L(G)$ is context-free. If G generates some string w containing x as a substring, then $w \in L(H)$. Hence $L(H) \neq \phi$ and thus R rejects and M accepts. Conversely, if G never generates a string w containing x as a substring, no any $w \in L(G)$. Since $L(H) = \phi$, R will accept and M will reject. As a result, M decides C .

4.28 Use the diagonalization method to obtain D . Let $\langle M_1 \rangle, \langle M_2 \rangle, \dots$ be the output of an enumerator for A . Let $\Sigma = \{s_1, s_2, \dots\}$ be a list of all strings over the alphabet. The algorithm for D is:
 $D =$ ”On input w :

1. Let i be the index of w on the list of all strings, that is, $s_i = w$.
2. Run $\langle M_i \rangle$ on input w .
3. If it accepts, *reject*. If it rejects, *accept*.”

D is a decider because each M_i is a decider. But D doesn't appear on the enumeration for A , because it differs from M_i on input s_i .

5.1 Suppose for a contradiction that EQ_{CFG} is decidable and assume the decider is N . We construct a decider M for $ALL_{CFG} = \{\langle G \rangle \mid G \text{ is a CFG and } L(G) = \Sigma^*\}$ as follows:
 $M =$ ”On input $\langle G \rangle$:

1. Construct a CFG H such that $L(H) = \Sigma^*$.
2. Run the decider N for EQ_{CFG} on $\langle G, H \rangle$.
3. if it accepts, *accept*. If it rejects, *reject*.

If $L(G) = \Sigma^*$, then $L(G) = L(H)$ and N will accept. Conversely, if $L(G) \neq \Sigma^*$, then $L(G) \neq L(H)$ and N will reject. M decides ALL_{CFG} assuming a decider N for EQ_{CFG} exists. Since we know ALL_{CFG} is undecidable, we have a contradiction; thus, EQ_{CFG} is undecidable.

5.3 Here is a match: $[\frac{ab}{abab}][\frac{ab}{abab}][\frac{aba}{b}][\frac{b}{a}][\frac{b}{a}][\frac{aa}{a}][\frac{aa}{a}]$.

5.4 No, it doesn't. For example, $\{a^n b^n c^n \mid n \geq 0\} \leq_m \{a, b\}$. The reduction first tests whether its input is a member of $\{a^n b^n c^n \mid n \geq 0\}$. If so, it outputs the string ab , and if not, it outputs the string a . Since the language $\{a^n b^n c^n \mid n \geq 0\}$ is decidable and not regular, it shows that the answer is no.

5.9 We show that $A_{TM} \leq_m T$ by mapping $\langle M, w \rangle$ to $\langle M' \rangle$ where M' is the following TM:
 $M' =$ ”On input x :

1. If $x \neq 01$ and $x \neq 10$ then *reject*.
2. If $x = 01$ then *accept*.

3. If $x=10$ simulates M on w . If M accepts w then *accept*; if M halts and rejects w then *reject*."

If $\langle M, w \rangle \in A_{TM}$ then M accepts w and $L(M') = \{01, 10\}$, so $\langle M' \rangle \in T$. Conversely, if $\langle M, w \rangle \notin A_{TM}$ then $L(M') = \{01\}$, so $\langle M' \rangle \notin T$. Therefore, $\langle M, w \rangle \in A_{TM} \Leftrightarrow \langle M' \rangle \in T$. Since A_{TM} is undecidable, then T is undecidable.

- 5.13 Let $USELESS_{TM} = \{\langle M \rangle \mid M \text{ is a TM with one or more useless states}\}$. Show that A_{TM} reduces to $USELESS_{TM}$. Assume for the sake of contradiction that TM R decides $USELESS_{TM}$. Construct TM S that uses R to decide A_{TM} . The new TM has a useless state exactly when M doesn't accept w . Doing so carefully can be messy. For convenience we use the universal TM.
 $S =$ "On input $\langle M, w \rangle$:

1. $T =$ "On input x :
 - (a) Replace x on the input by the string $\langle M, w \rangle$.
 - (b) Run the universal TM U that is described after the statement of Theorem 4.11 to simulate $\langle M, w \rangle$. Note that this TM was designed to use all of its states.
 - (c) If U accepts, enter a special state q_A and *accept*."
2. Run R on $\langle T \rangle$ to determine whether T has any useless states.
3. If R rejects, M accepts w , therefore *accept*. Otherwise, *reject*."

If M accepts w , then T enters all states, but if M doesn't accept w , then T avoids q_A . So T has a useless state- namely q_A - if and only if M doesn't accept w . Since A_{TM} is undecidable, $USELESS_{TM}$ is undecidable.

- 5.17 The *PCP* over a unary alphabet is decidable. We describe a TM M that decides unary *PCP*. Given a unary *PCP* instance

$$P = \{[\frac{1^{a_1}}{1^{b_1}}], \dots, [\frac{1^{a_n}}{1^{b_n}}]\}$$

$M =$ "On input $\langle P \rangle$:

1. Check if $a_i = b_i$ for some i . If so, *accept*.
2. Check if there exist i, j such that $a_i > b_i$ and $a_j < b_j$. If so, *accept*. Otherwise, *reject*."

In the first stage, M first checks for a single domino which forms a match. In the second stage, M looks for two dominos which form a match. If it finds such a pair, it can construct a match by picking $(b_j - a_i)$ copies of the i^{th} domino, putting them together with $(a_i - b_i)$ copies of the j^{th} domino. This construction has $a_i(b_j - a_j) + a_j(a_i - b_i) = a_i b_j - a_j b_i$ 1s on the top, and $b_i(b_j - a_j) + b_j(a_i - b_i) = a_i b_j - a_j b_i$ 1s on the bottom. If neither stages of M accept, the problem instance contains dominos with all the upper parts having

more (or less) 1s than the lower parts. In such a case no match can exist. Therefore M rejects.

5.18 We construct a computable function that maps a PCP instance to a PCP instance over a binary alphabet ($BPCP$). Therefore, $BPCP$ is undecidable. For any PCP instance A over alphabet $\{a_1, \dots, a_n\}$, the function encodes each character a_i into binary as 10^i . The function is a mapping reduction from PCP to $BPCP$. For any P in PCP , the converted instance P' is in $BPCP$ because a match for P becomes a match for P' using the same set of dominos after conversion. Conversely, if P' is in $BPCP$, P is in PCP , because we can uniquely decode a match in P' to a match in P . Therefore, the function is a mapping reduction from PCP to $BPCP$.

5.19 Any match for $SPCP$ starts with a domino that has two equal strings, and therefore is a match all by itself. So we need only check whether the input contains a domino that has two equal strings. If so, *accept*; otherwise, *reject*.

5.21 We reduce PCP to $AMBIG_{CFG}$, thereby proving that $AMBIG_{CFG}$ is undecidable. We use the construction given in the hint in the text. We show that P has a solution if and only if the CFG is ambiguous.

If P has a match $t_{i_1}t_{i_2} \dots t_{i_l} = b_{i_1}b_{i_2} \dots b_{i_l}$, the string

$$t_{i_1}t_{i_2} \dots t_{i_l}a_{i_l} \dots a_{i_2}a_{i_1} = b_{i_1}b_{i_2} \dots b_{i_l}a_{i_l} \dots a_{i_2}a_{i_1}$$

has two different leftmost derivations, one from T and one from B . Hence the CFG is ambiguous.

Coversely, if the CFG is ambiguous, some string w has multiple leftmost derivations. All generated strings have the form $w = w_{start}a_{i_l} \dots a_{i_2}a_{i_1}$ where w_{start} contains only symbols from P 's alphabet. Notice that once we choose the first step in the derivation of w (either $S \rightarrow T$ or $S \rightarrow B$), the following steps in the derivation are uniquely determined by the sequence $a_{i_l} \dots a_{i_2}a_{i_1}$. Therefore w has at most two leftmost derivations:

1. $S \rightarrow T \rightarrow t_{i_1}Ta_{i_1} \rightarrow t_{i_1}t_{i_2}Ta_{i_2}a_{i_1} \rightarrow \dots \rightarrow t_{i_1}t_{i_2} \dots t_{i_l}a_{i_l} \dots a_{i_2}a_{i_1}$, and
2. $S \rightarrow B \rightarrow b_{i_1}Ba_{i_1} \rightarrow b_{i_1}b_{i_2}Ba_{i_2}a_{i_1} \rightarrow \dots \rightarrow b_{i_1}b_{i_2} \dots b_{i_l}a_{i_l} \dots a_{i_2}a_{i_1}$

If w is ambiguous, $t_{i_1}t_{i_2} \dots t_{i_l}a_{i_l} \dots a_{i_2}a_{i_1} = b_{i_1}b_{i_2} \dots b_{i_l}a_{i_l} \dots a_{i_2}a_{i_1}$, and therefore $t_{i_1}t_{i_2} \dots t_{i_l} = b_{i_1}b_{i_2} \dots b_{i_l}$. Thus, the initial PCP instance P has a match.